AI Multimodal Detection System

Project Introduction and Technical Roadmap for YOLOE-11 Open-Vocabulary Detection

Document version	v1.0
Last updated	31 August 2026
Default model	YOLOE-11l-seg (open vocabulary, high accuracy)

1. Project Overview

This project is a Python desktop AI vision application for real-time object detection from a camera or local video. Its core capability is open-vocabulary detection: users can enter natural-language target phrases at runtime, such as 'milk bottle', 'red backpack', or 'mobile phone'. The system uses a vision-language model to locate zero-shot targets instead of being limited to a fixed class list in a conventional detector.

The project is designed for usability and maintainability. The UI, model inference, video worker, and common utilities are separated into dedicated modules, while CUDA inference, error diagnostics, low-light enhancement, and visual prompting remain available as features or extension points.

2. Implemented Capabilities

Module	Current capability	Verification status
Open text prompts	Accepts multiple prompts separated by commas, semicolons, or line breaks, including phrase-level object descriptions.	A real local-image inference was completed with the 'person' prompt on YOLOE-11l.
High-accuracy model	Uses YOLOE-11l-seg by default, with YOLOE-11s / 11m / 11l selectable.	The 11l weights were loaded successfully in a CUDA environment.
GPU acceleration	Supports CPU or CUDA and checks CUDA availability during startup.	Verified on an RTX 5070 Laptop GPU where torch.cuda.is_available() returned true.
Video input	Supports Camera 0 and MP4/AVI/MOV/MKV files; DirectShow is preferred for Windows cameras.	UI and acquisition/error-handling code are complete; continuous streams were not acceptance-tested across every source.
Black-screen and error handling	Displays the first camera frame before model warm-up and shows inference exceptions in a	Validated through error diagnosis and UI construction checks.

	dialog.	
Visual prompting	Lets users draw a target box on a reference image and applies HSV color-histogram similarity as a second-stage filter.	UI and algorithm flow are implemented; no real-scene end-to-end benchmark has yet been completed.
Low-light enhancement	Loads Zero-DCE ONNX when available and falls back to CLAHE when it is absent.	The CLAHE fallback is implemented; the Zero-DCE weight branch awaits validation with a supplied model.

3. Software Structure and Runtime Flow

File	Responsibility
new.py	Application entry point and controller for model loading, source selection, detection lifecycle, and user-facing errors.
ui.py	PySide6 layout construction and the reference-image selection canvas.
engines.py	YOLOE-11 open-vocabulary detector plus the Zero-DCE/CLAHE low-light enhancement engine.
video_worker.py	QThread-based video acquisition, preview, inference, and status signals.
app_utils.py	Reusable helpers, including prompt parsing.
run_app.bat	Launches the application with the verified pytorch Conda environment.

- Launch: use run_app.bat, or activate the pytorch Conda environment and run python new.py.
- Load: select a model scale and device, then click 'Apply Settings / Download and Load YOLOE-11 Model'.
- Detect: enter one or more prompts and start the camera or video stream. The first frame appears before model warm-up; detection boxes and confidences follow.

4. Key Technologies

Technology	Role in the project
YOLOE-11	An open-vocabulary detection and instance-segmentation model. It extends conventional YOLO beyond fixed categories through text, visual, and prompt-free mechanisms; this project uses the text-prompt path.
CLIP / vision-language alignment	Encodes a user's natural-language query into text embeddings so runtime concepts such as 'milk bottle' can participate in detection.
PyTorch + CUDA	Loads YOLOE weights and runs inference on NVIDIA GPUs.
Ultralytics	Provides model management, weight loading, prediction, and post-processing interfaces for YOLOE.
OpenCV	Handles cameras and video files, color conversion, box drawing, HSV histogram comparison, and CLAHE enhancement.
PySide6 / Qt	Builds the desktop GUI and isolates video/inference work from the UI thread with QThread.

ONNX Runtime	Provides the optional inference backend for the Zero-DCE low-light enhancement model.

5. Current Boundaries and Usage Guidance

- Open vocabulary does not mean every concept will be detected reliably. Clear images, common visible objects, and specific correctly spelled English prompts generally produce the best results.
- The first use of text prompts requires the official CLIP text-encoder dependency. The project has fixed the prior error caused by installing an unrelated clipboard-manager package with the same name, clip.
- YOLOE-11l prioritizes accuracy and therefore requires more VRAM and inference time. Choose YOLOE-11s when real-time responsiveness is more important.
- The current visual-prompt path uses color-histogram filtering and is sensitive to illumination, background, and material changes. It should not be treated as deep visual retrieval.

6. Future Optimization Directions

Direction	Recommended work	Expected value
Visual-prompt upgrade	Adopt YOLOE native visual prompting (SAVPE) or CLIP/DINO feature retrieval instead of relying solely on HSV histograms.	More robust matching under illumination changes, occlusion, and specific brand/part targets.
Prompt-free discovery	Integrate YOLOE prompt-free weights and its built-in vocabulary to propose classes when the user does not know what is present.	Extends the workflow from finding a named target to exploring a scene.
Video tracking	Use ByteTrack or BoT-SORT to assign stable IDs to detections.	Reduces frame-to-frame jitter and enables counting, trajectories, and alerts.
Auto-labeling	Export frames, boxes, and labels to YOLO / COCO / VOC datasets with a human review step.	Accelerates data collection and domain adaptation.
Domain fine-tuning	Collect task-specific data and fine-tune YOLOE / YOLO11 for detection or segmentation.	Improves accuracy for small targets, unusual angles, and industrial parts.
Performance engineering	Add frame skipping, asynchronous queues, mixed precision, TensorRT/ONNX export, and multi-camera resource management.	Reduces latency and increases throughput.
Evaluation and explainability	Add inference logs, FPS/VRAM panels, dataset evaluation, confusion analysis, and replay.	Provides measurable evidence for model selection and iteration.

7. Recommended Next-Stage Acceptance Tests

- Camera validation: run sustained CPU and CUDA sessions and record startup time, FPS, VRAM use, and error recovery behavior.
- Prompt-set evaluation: prepare at least 30 target phrases across people, containers, tools, electronics, and small objects.
- Video and low-light tests: compare raw video, CLAHE, and Zero-DCE on local files and practical low-light scenes.
- Visual-prompt acceptance: prepare samples across lighting and backgrounds, then compare recall and false positives before and after the planned upgrade.

8. Reference Papers and Resources

[1] Wang, A. et al. YOLOE: Real-Time Seeing Anything. arXiv:2503.07465, 2025. https://arxiv.org/abs/2503.07465

[2] Radford, A. et al. Learning Transferable Visual Models from Natural Language Supervision. ICML, 2021, pp. 8748-8763. https://mlanthology.org/icml/2021/radford2021icml-learning/

[3] Cheng, T. et al. YOLO-World: Real-Time Open-Vocabulary Object Detection. CVPR, 2024. https://arxiv.org/abs/2401.17270

[4] Guo, C. et al. Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement. CVPR, 2020, pp. 1780-1789. https://openaccess.thecvf.com/content_CVPR_2020/html/Guo_Zero-Reference_Deep_Curve_Estimation_for_Low-Light_Image_Enhancement_CVPR_2020_paper.html

[5] Ultralytics Documentation. YOLOE Open-Vocabulary Detection & Segmentation. https://docs.ultralytics.com/models/yoloe/

AI 多模态检测系统

YOLOE-11 开放词汇目标检测项目介绍与技术路线

文档版本	v1.0
更新日期	2026 年 8 月 31 日
当前默认模型	YOLOE-11l-seg（开放词汇、高精度）

1. 项目概述

本项目是一个基于 Python 的桌面端 AI 视觉应用，用于在摄像头或本地视频中进行实时目标检测。当前核心能力是开放词汇检测：用户可在运行时输入自然语言目标短语，例如 “milk bottle”“red backpack” 或 “mobile phone”，系统利用视觉-语言模型完成零样本目标定位，而不是受限于传统检测模型的固定类别表。

项目以可用性和可维护性为目标：将界面、模型推理、视频线程与通用工具拆分为独立模块，同时保留 CUDA 推理、错误诊断、低照度增强与视觉提示词等功能入口。

2. 当前已实现功能

模块	当前能力	验证状态
开放文本提示词	接受逗号、分号或换行分隔的多个提示词；支持短语级目标描述。	已在 YOLOE-11l 上用 person 提示词和本地图片完成真实推理。
高精度模型	默认使用 YOLOE-11l-seg；可切换 YOLOE-11s / 11m / 11l。	已在 CUDA 环境成功加载 11l 权重。
GPU 加速	CPU / CUDA 可选；启动时检查 CUDA 是否可用。	已确认 RTX 5070 Laptop GPU 环境中 torch.cuda.is_available() 为真。
视频输入	支持 Camera 0 和 MP4/AVI/MOV/MKV 本地文	UI 与采集/错误处理代码已完

	件；Windows 摄像头优先 DirectShow。	成；持续视频流未作为发布验收项逐源测试。
黑屏与异常诊断	首帧在模型预热前显示；推理异常弹窗显示具体异常。	已通过异常定位与 UI 构建验证。
视觉提示词	参考图拖拽框选目标后，使用 HSV 颜色直方图做二次相似度筛选。	界面与算法流程已实现；尚未做真实场景端到端评测。
暗光增强	优先加载 Zero-DCE ONNX；缺失时退化为 CLAHE。	CLAHE 回退逻辑已实现；Zero-DCE 权重分支待提供模型后验证。

3. 软件结构与运行流程

文件	职责
new.py	应用入口与控制器：模型加载、输入源选择、启动/停止检测、错误提示。
ui.py	PySide6 界面构建与参考图片框选画布。
engines.py	YOLOE-11 开放词汇检测引擎与 Zero-DCE/CLAHE 暗光增强引擎。
video_worker.py	QThread 视频采集、预览、推理与状态信号。
app_utils.py	提示词解析等可复用工具。
run_app.bat	使用已验证的 pytorch Conda 环境启动程序。

- 启动：使用 run_app.bat 或在 pytorch Conda 环境中运行 python new.py。
- 加载：选择模型规模与设备，点击“应用设置 / 下载并加载 YOLOE-11 模型”。
- 检测：输入一个或多个文本提示词，启动摄像头/视频流；首帧先显示，随后叠加检测框和置信度。

4. 关键技术

技术	在项目中的作用
YOLOE-11	开放词汇检测与实例分割模型。通过文本、视觉或无提示词机制扩展传统 YOLO 的固定类别限制；本项目使用文本提示词检测路径。
CLIP / 视觉-语言对齐	将用户输入的自然语言转为文本嵌入，使 “milk bottle” 等运行时类别可参与

	检测。
PyTorch + CUDA	加载 YOLOE 权重并调用 NVIDIA GPU 加速推理。
Ultralytics	提供 YOLOE 模型管理、权重加载、预测与后处理接口。
OpenCV	摄像头与视频读取、颜色空间转换、绘制框、HSV 直方图相似度和 CLAHE 图像增强。
PySide6 / Qt	构建桌面 GUI，并以 QThread 将视频/推理从界面线程中分离。
ONNX Runtime	为可选的 Zero-DCE ONNX 暗光增强模型提供推理后端。

5. 已知边界与使用建议

- “开放词汇”并不等于任何概念都能稳定识别。清晰画面、常见可见物体、具体且拼写正确的英文短语通常效果更好。
- 文本提示词的首次使用需要官方 CLIP 文本编码依赖；项目已修复误装同名 clipboard clip 包导致的属性错误。
- YOLOE-11l 追求精度，需要更多显存与推理时间；实时性优先时应选择 YOLOE-11s。
- 当前视觉提示词使用颜色直方图二次筛选，对光照、背景和材质变化敏感，不应视为深度视觉检索能力。

6. 后续优化方向

方向	建议工作	预期价值
视觉提示词升级	接入 YOLOE 原生视觉提示词（SAVPE）或 CLIP/DINO 特征检索，替换单纯 HSV 直方图。	提升对颜色变化、遮挡和特定品牌/部件目标的稳健性。
无提示词发现	接入 YOLOE prompt-free 权重与内置大词表，为未知画面提供候选类别。	从“找指定目标”扩展为“探索画面中有什么”。
视频追踪	使用 ByteTrack 或 BoT-SORT 为检测目标分配稳定 ID。	减少逐帧框抖动，支持计数、轨迹和告警。
自动标注	将视频帧、检测框和类别导出为 YOLO / COCO / VOC 数据集格式，并加入人工复核。	加速特定业务场景的数据积累与微调。
领域微调	围绕实际对象采集数据，使用 YOLOE / YOLO11 进行检测或分割微调。	提高小目标、特殊角度、工业零件等领域场景精度。

性能工程	加入帧跳过、异步队列、半精度、TensorRT/ONNX 导出与多摄像头资源管理。	降低延迟并提高吞吐量。
可解释与评估	增加推理日志、FPS/显存面板、数据集评测、混淆分析与回放。	使模型选择与迭代有量化依据。

7. 建议的下一阶段验收

- 摄像头实测：分别在 CPU 与 CUDA 下连续运行，记录启动耗时、FPS、显存占用与异常恢复情况。
- 提示词集评测：建立不少于 30 个目标短语的测试表，覆盖人物、容器、工具、电子产品和小目标。
- 视频与弱光测试：用本地视频和实际低照度场景比较原始画面、CLAHE 与 Zero-DCE 的检测效果。
- 视觉提示词验收：准备多光照、多背景样本，确认升级前后相似目标检索的召回率和误检率。

8. 参考论文与资料

[1] Wang, A. et al. YOLOE: Real-Time Seeing Anything. arXiv:2503.07465, 2025. https://arxiv.org/abs/2503.07465

[2] Radford, A. et al. Learning Transferable Visual Models from Natural Language Supervision. ICML, 2021, pp. 8748-8763. https://mlanthology.org/icml/2021/radford2021icml-learning/

[3] Cheng, T. et al. YOLO-World: Real-Time Open-Vocabulary Object Detection. CVPR, 2024. https://arxiv.org/abs/2401.17270

[4] Guo, C. et al. Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement. CVPR, 2020, pp. 1780-1789. https://openaccess.thecvf.com/content_CVPR_2020/html/Guo_Zero-Reference_Deep_Curve_Estimation_for_Low-Light_Image_Enhancement_CVPR_2020_paper.html

[5] Ultralytics Documentation. YOLOE Open-Vocabulary Detection & Segmentation. https://docs.ultralytics.com/models/yoloe/